
COMPASS-KEM

Design Specifications and Supporting Documentation

Version: 1.2
Date: June 2026

Author: Geng Wang, Zhedong Wang, Wenwen Xia, Ningyue Zhang,
Wenyu He, Xinyang Geng, Fei Duan, Weidan Ji, Jihong Li
Organization: Laboratory of Cryptology and Computer Security,
Shanghai Jiao Tong University; Shanghai Zhixun Crypto Co. Ltd.
Status: Draft Specification

Contents

1	Introduction	1
2	Algorithm Description	1
2.1	Symbols and Notations	1
2.2	Interpreting the Pseudocode	3
2.2.1	Data Types	3
2.2.2	Arithmetic With Arrays of Integers	4
2.2.3	Representations of Algebraic Objects	4
2.2.4	Arithmetic With Polynomials and NTT Representations	5
2.2.5	Matrices and Vectors	5
2.2.6	Arithmetic With Matrices and Vectors	5
2.2.7	Applying Algorithms to Arrays, Examples	6
2.3	Basic Operations	7
2.3.1	Hashing	7
2.3.2	NTT domain representation	7
2.3.3	Compression and Decompression	8
2.4	COMPASS-PKE Algorithms	9
2.4.1	Key Generation (KeyGen)	9
2.4.2	Encryption (Encrypt)	10
2.4.3	Decryption (Decrypt)	11
2.5	Main Algorithms	11
2.5.1	Key Generation	12
2.5.2	Encapsulation	12
2.5.3	Decapsulation	12
3	Design Rationale	13
3.1	Security Foundation	13
3.2	Parameter Choices	15
3.3	Decryption Failure Analysis	15
3.4	Other Considerations	16
3.5	Dissimilarity Claim	17
4	Security Claim and Analysis	17
4.1	Theoretical Security	17
4.1.1	IND-CPA Security	17
4.1.2	IND-CCA Security in Classical Random Oracle	18
4.1.3	IND-CCA Security in Quantum Random Oracle	19
4.2	Concrete Security	19
4.2.1	Primal Attack	19
4.2.2	Dual Attack	21
5	Performances	21
5.1	Performance Analysis	22
5.1.1	Implementation Considerations and Tradeoffs	22
5.1.2	Performance Scaling Across Security Levels	23
5.2	Reference and Optimized Implementations	23

5.2.1	Portable ANSI C Baseline	23
5.2.2	AVX2 Optimized Implementation	24
6	Feature Statement	25

1. Introduction

This specification defines COMPASS-KEM, a part of the COMcompact Post-quAntum Security Suite, which is a post-quantum Key Encapsulation Mechanism (KEM) based on the Module Learning-with-Rounding (MLWR) problem. The algorithm is designed to be secure against large-scale quantum computers, and supports four security categories, corresponding to 128-bit (80-bit quantum), 256-bit (128-bit quantum), 384-bit (192-bit quantum), and 512-bit (256-bit quantum) security.

Key advantages of the algorithm include:

- **Solid Foundation from Provable Security:** The security of scheme is based on the decisional MLWR problem over prime moduli and a variant of this problem. Earlier MLWR-based scheme such as SABER [DKRV18] in the NIST standardization process has its security based on MLWR with power-of-2 moduli, which lacks security reduction from standard lattice hard problems. Instead, MLWR with prime moduli allows a reduction from MLWE, thereby inheriting worst-case complexity over lattice problems. Additionally, prime moduli allow the usage of NTT that greatly increases the efficiency.

We also need to point out that, the security proof in the original paper of SABER [DKRV18] is flawed. We fix this issue by introducing a variant of MLWR problem in our security proof, which shares a similar reduction from MLWE, thus does not affect the provable security of the scheme.

- **Efficiency:** Compared with MLWE-based schemes such as ML-KEM, the noise sampling steps are replaced by a fast-to-implement compression algorithm. By public key compression, the scheme enjoys smaller public key sizes ($\leq 84\%$) compared to ML-KEM under same security level. The ciphertext size under 256-bit security level is also smaller than ML-KEM. The cost of public key compression is that the NTT operation on the compressed public key must be performed in the encapsulation and decapsulation algorithms, instead of in the key generation algorithm. This slightly increases the time costs of encapsulation and decapsulation algorithms ($\leq 5\%$).
- **Free from Intellectual Property Issues:** By avoiding the usage of error terms as in ML-KEM, the scheme is made to be free from the intellectual property issues met by ML-KEM or other LWE-based schemes.

2. Algorithm Description

2.1 Symbols and Notations

The primary parameters of the algorithm are defined as follows:

n Degree of the modulus polynomial.

q Integer modulus.

ζ Denotes a primitive n -th root of unity modulo q .

$\mathbb{B}$ The set $\{0, 1, \dots, 255\}$ of unsigned 8-bit integers (bytes).

$\mathbb{Z}$ The set of integers.

- $\mathbb{Z}_m$ The ring of integers modulo m (i.e., the set $\{0, 1, \dots, m-1\}$ equipped with the operations of addition and multiplication modulo m).
- $\mathbb{Z}_m^n$ The set of n -tuples over $\mathbb{Z}_m$ equipped with $\mathbb{Z}_m$ -module structure. As a data type, this is the set of length- n arrays whose entries are in $\mathbb{Z}_m$.
- R The ring $\mathbb{Z}[X]/(X^n + 1)$.
- R_q The ring $\mathbb{Z}_q[X]/(X^n + 1)$ consisting of polynomials of the form $f = f_0 + f_1X + \dots + f_{n-1}X^{n-1}$, where $f_j \in \mathbb{Z}_q$ for all j .
- T_q The image of R_q under the number-theoretic transform. Its elements are called “NTT representations” of polynomials in R_q .
- $\mathcal{D}_\eta(R_q)$ A certain distribution of polynomials in R_q with small coefficients, from which noise is sampled. The distribution is parameterized by $\eta \in \{2, 3, 4\}$.
- S^* If S is a set, this denotes the set of finite-length tuples (or arrays) of elements from S , including the empty tuple.
- S^k If S is a set, this denotes the set of k -tuples (or length- k arrays) of elements from S .
- f_j The coefficient of X^j of a polynomial $f = f_0 + f_1X + \dots + f_{n-1}X^{n-1} \in R_q$.
- $\hat{f}$ The element of T_q that is equal to the NTT representation of a polynomial $f \in R_q$.
- $\mathbf{v}^T, \mathbf{A}^T$ The transpose of a row or column vector $\mathbf{v}$. In general, the transpose of a matrix $\mathbf{A}$.
- $\circ$ Denotes linear-algebraic composition with coefficients in R_q or T_q (e.g., $\mathbf{A} \circ \mathbf{v}$).
- $A \parallel B$ The concatenation of two arrays or bit strings A and B .
- $B[i]$ The entry at index i in the array B . All arrays have indices that begin at zero.
- $B[k : m]$ The subarray $(B[k], B[k+1], \dots, B[m-1])$ of the array B .
- $|B|$ If B is a number, this denotes the absolute value of B . If B is an array, this denotes its length.
- $\lceil x \rceil$ The ceiling of x (the smallest integer greater than or equal to x).
- $\lfloor x \rfloor$ The floor of x (the largest integer less than or equal to x).
- $\lceil x \rceil$ Rounding of x to the nearest integer. If $x = y + 1/2$ for $y \in \mathbb{Z}$, then $\lceil x \rceil = y + 1$.
- $r \bmod m$ The unique integer $r' \in \{0, 1, \dots, m-1\}$ such that $m \mid r - r'$.
- $\text{BitRev}_7(r)$ Bit reversal of an integer r .
- $s \leftarrow x$ In pseudocode, variable s is assigned the value of expression x .
- $s \xleftarrow{\$} \mathbb{B}^\ell$ In pseudocode, s is an array of ℓ fresh random bytes from an approved random byte generator.
- $\perp$ A symbol indicating failure or the lack of output from an algorithm.

k : number of MLWR samples;
 l : dimension of MLWR secret;
 d : Number of compression bits in the public key.
 d_1, d_2 : Numbers of compression bits in the ciphertext.
 η_s Distribution parameter for private key.
 η_2 Distribution parameter for encryption randomness.
 β_η Centered Binomial Distribution with parameter η .
 $\odot$ Multiplication in the NTT domain.
 $\bmod q$ Modular reduction with result in $\mathbb{Z}_q = \{0, \dots, q-1\}$.

2.2 Interpreting the Pseudocode

This section outlines the conventions of the pseudocode used to describe the algorithms in this documentation. All algorithms are understood to have access to two global integer constants: $n = 256$ and $q = 3329$, as well as $n = 512$ and $q = 7681$. There are also global integer variables: n , q , $k \times l$, η_s , η_2 , (d, d_1, d_2) . All other variables are local. The five global variables are set to particular values when a parameter set is selected (see Section 3.2).

When algorithms in this documentation invoke other algorithms as subroutines, all arguments (i.e., inputs) are passed by value. In other words, a copy of the inputs is created, and the subroutine is invoked with the copy. There is no “passing by reference.”

Pseudocode assignments are performed using the symbol “ $\leftarrow$.” For example, the statement $z \leftarrow y$ means that the variable z is assigned the value of variable y . Pseudocode comparisons are performed via the symbol “ $==$.” For example, the expression $x == w$ is a boolean value that is TRUE if and only if the variables x and w have the same value.

2.2.1 Data Types

For variables that represent the input or output of an algorithm, the data type (e.g., bit, byte, array of bits) will be explicitly described at the start of the algorithm. For most local variables in the pseudocode, the data type is easily deduced from context. For all other variables, the data type will be declared in a comment. In a single algorithm, the data type of a variable is determined the first time that the variable is used and will not be changed. Variable names can and will be reused across different algorithms, including with different data types.

In addition to standard atomic data types (e.g., bits, bytes) and data structures (e.g., arrays), integers modulo m (i.e., elements of $\mathbb{Z}_m$) will also be used as an abstract data type. It is implicit that reduction modulo m takes place whenever an assignment is made to a variable in $\mathbb{Z}_m$. For example, for $z \in \mathbb{Z}_m$ and integers x and y , the statement

$$z \leftarrow x + y$$

means that z is assigned the value $x+y \bmod m$. The pseudocode is agnostic regarding how an integer modulo m is represented in actual implementations or how modular reduction is computed.

2.2.2 Arithmetic With Arrays of Integers

This documentation makes extensive use of arrays of integers modulo m (i.e., elements of $\mathbb{Z}_m^\ell$). In a typical case, the relevant values are $m = q = 3329$, $\ell = n = 256$ or $m = q = 7681$, $\ell = n = 512$. Arithmetic with arrays in $\mathbb{Z}_m^\ell$ will be done as follows. Let $a \in \mathbb{Z}_m$ and $X, Y \in \mathbb{Z}_m^\ell$. The statements

$$Z \leftarrow a \cdot X \tag{2.2}$$

$$W \leftarrow X + Y \tag{2.3}$$

will result in two arrays $Z, W \in \mathbb{Z}_m^\ell$, with the property that $Z[i] = a \cdot X[i]$ and $W[i] = X[i] + Y[i]$ for all i . Multiplication of arrays in $\mathbb{Z}_m^\ell$ will only be meaningful when $m = q$ and $\ell = n$, in which case it corresponds to multiplication in a particular ring.

2.2.3 Representations of Algebraic Objects

An essential operation in COMPASS-KEM is the number-theoretic transform (NTT), which maps a polynomial f in a certain ring R_q to its “NTT representation” $\hat{f}$ in an isomorphic ring T_q . The rings R_q and T_q and the NTT are discussed in detail. This documentation will represent elements of R_q and T_q in pseudocode using arrays of integers modulo q as follows.

An element f of R_q is a polynomial of the form

$$f = f_0 + f_1X + \cdots + f_{n-1}X^{n-1} \in R_q \tag{2.4}$$

and will be represented in pseudocode by the array

$$(f_0, f_1, \dots, f_{n-1}) \in \mathbb{Z}_q^n, \tag{2.5}$$

whose entries contain the coefficients of f . Overloading notation, the array in (2.5) will also be denoted by f . The i -th entry of the array f will thus contain the i -th coefficient of the polynomial f (i.e., $f[i] = f_i$).

An element (sometimes called “NTT representation”) $\hat{g}$ of T_q is a tuple of $n/2$ polynomials, each of degree at most one. Specifically,

$$\hat{g} = (\hat{g}_{0,0} + \hat{g}_{0,1}X, \hat{g}_{1,0} + \hat{g}_{1,1}X, \dots, \hat{g}_{n/2-1,0} + \hat{g}_{n/2-1,1}X) \in T_q. \tag{2.6}$$

Such an algebraic object will be represented in pseudocode by the array

$$(\hat{g}_{0,0}, \hat{g}_{0,1}, \hat{g}_{1,0}, \hat{g}_{1,1}, \dots, \hat{g}_{n/2-1,0}, \hat{g}_{n/2-1,1}) \in \mathbb{Z}_q^n. \tag{2.7}$$

Overloading notation, the array in (2.7) will also be denoted by $\hat{g}$. In this case, the mapping between array entries and coefficients is $\hat{g}[2i] = \hat{g}_{i,0}$ and $\hat{g}[2i+1] = \hat{g}_{i,1}$ for $i \in \{0, 1, \dots, n/2-1\}$.

Converting between a polynomial $f \in R_q$ and its NTT representation $\hat{f} \in T_q$ will be done via the algorithms **NTT** and **INTT**. These algorithms act on arrays of coefficients, as described above, and satisfy $\hat{f} = \text{NTT}(f)$ and $f = \text{INTT}(\hat{f})$.

2.2.4 Arithmetic With Polynomials and NTT Representations

The algebraic operations of addition and scalar multiplication in R_q and T_q are done coordinate-wise. For example, if $a \in \mathbb{Z}_q$ and $f \in R_q$, the i -th coefficient of the polynomial $a \cdot f \in R_q$ is equal to $a \cdot f_i \bmod q$. In pseudocode, elements of both R_q and T_q are represented by coefficient arrays (i.e., elements of $\mathbb{Z}_q^n$). The algebraic operations of addition and scalar multiplication are thus performed by addition and scalar multiplication of the corresponding coefficient arrays, as in (2.3) and (2.2). For example, the addition of two NTT representations in pseudocode is performed by a statement of the form $\hat{h} \leftarrow \hat{f} + \hat{g}$, where $\hat{h}, \hat{f}, \hat{g} \in \mathbb{Z}_q^n$ are coefficient arrays.

The algebraic operations of multiplication in R_q and T_q are treated as follows. For efficiency reasons, multiplication in R_q will not be used. The algebraic meaning of multiplication in T_q is discussed in Section 2.3.2 for more details. Specifically, for $f, g \in R_q$, let $\hat{f} = \text{NTT}(f), \hat{g} = \text{NTT}(g) \in \mathbb{Z}_q^n$ are a pair of arrays, interpreted as elements in T_q , we write

$$\hat{h} \leftarrow \hat{f} \odot \hat{g} \quad (2.8)$$

and for $h = \text{INTT}(\hat{h})$, there is $h = f \cdot g$ under the ring R_q .

2.2.5 Matrices and Vectors

In addition to arrays of integers modulo q , the pseudocode will also make use of arrays whose entries are themselves elements of $\mathbb{Z}_q^n$. For example, an element $\mathbf{v} \in (\mathbb{Z}_q^n)^3$ will be a length-three array whose entries $\mathbf{v}[0]$, $\mathbf{v}[1]$, and $\mathbf{v}[2]$ are themselves elements of $\mathbb{Z}_q^n$ (i.e., arrays). One can think of each of these entries as representing a polynomial in R_q so that $\mathbf{v}$ itself represents an element of the module R_q^3 .

When arrays are used to represent matrices and vectors whose entries are elements of R_q , they will be denoted with bold letters (e.g., $\mathbf{v}$ for vectors and $\mathbf{A}$ for matrices). When arrays are used to represent matrices and vectors whose entries are elements of T_q , they will be denoted with a “hat” (e.g., $\hat{\mathbf{v}}$ and $\hat{\mathbf{A}}$). Unless an explicit transpose operation is performed, it is understood that vectors are column vectors. One can then view vectors as the special case of matrices with only one column.

Converting between matrices over R_q and matrices over T_q will be done coordinate-wise. For example, if $\mathbf{v} \in (\mathbb{Z}_q^n)^k$, then the statement

$$\hat{\mathbf{v}} \leftarrow \text{NTT}(\mathbf{v}) \quad (2.9)$$

will result in $\hat{\mathbf{v}} \in (\mathbb{Z}_q^n)^k$ such that $\hat{\mathbf{v}}[i] = \text{NTT}(\mathbf{v}[i])$ for all i . This involves running NTT a total of k times.

2.2.6 Arithmetic With Matrices and Vectors

The following describes how to perform arithmetic with matrices over R_q and T_q with vectors as a special case.

Addition and scalar multiplication are performed coordinate-wise, so the addition of matrices over R_q and T_q is straightforward. In the case of T_q , scalar multiplication is done via (2.8). For example, if $\hat{f} \in \mathbb{Z}_q^n$ and $\hat{\mathbf{u}}, \hat{\mathbf{v}} \in (\mathbb{Z}_q^n)^k$, then

$$\hat{\mathbf{w}} \leftarrow \hat{f} \odot \hat{\mathbf{u}} \quad (2.10)$$

$$\hat{\mathbf{z}} \leftarrow \hat{\mathbf{u}} + \hat{\mathbf{v}} \quad (2.11)$$

will result in $\hat{\mathbf{w}}, \hat{\mathbf{z}} \in (\mathbb{Z}_q^n)^k$ satisfying $\hat{\mathbf{w}}[i] = \hat{f} \odot \hat{\mathbf{u}}[i]$ and $\hat{\mathbf{z}}[i] = \hat{\mathbf{u}}[i] + \hat{\mathbf{v}}[i]$ for all i . Here, the multiplication and addition of individual entries are performed using the appropriate arithmetic for coefficient arrays of elements of T_q (i.e., as in (2.3)).

It will also be necessary to multiply matrices with entries in T_q , which is done by using standard matrix multiplication with the base-case multiplication (i.e., multiplication of individual entries) being multiplication in T_q . If $\hat{\mathbf{A}}$ and $\hat{\mathbf{B}}$ are two matrices with entries in T_q , their matrix product will be denoted by $\hat{\mathbf{A}} \odot \hat{\mathbf{B}}$. Some example pseudocode statements involving matrix multiplication are given in (2.12), (2.13), and (2.14). In these examples, $\hat{\mathbf{A}}$ is a $k \times k$ matrix, while $\hat{\mathbf{u}}$ and $\hat{\mathbf{v}}$ are vectors of length k . All three of these objects are represented in pseudocode by arrays: a $k \times k$ array for $\hat{\mathbf{A}}$ and length- k arrays for $\hat{\mathbf{u}}$ and $\hat{\mathbf{v}}$. The entries of $\hat{\mathbf{A}}$, $\hat{\mathbf{u}}$, and $\hat{\mathbf{v}}$ are elements of $\mathbb{Z}_q^n$. In (2.12) and (2.13), the pseudocode statement on the left produces a new length- k vector whose entries are specified on the right. In (2.14), the pseudocode statement on the left computes a dot product. The result is in the base ring (i.e., T_q) and is represented by an element $\hat{z}$ of $\mathbb{Z}_q^n$.

$$\hat{\mathbf{w}} \leftarrow \hat{\mathbf{A}} \odot \hat{\mathbf{u}} \quad \hat{\mathbf{w}}[i] = \sum_{j=0}^{k-1} \hat{\mathbf{A}}[i, j] \odot \hat{\mathbf{u}}[j] \quad (2.12)$$

$$\hat{\mathbf{y}} \leftarrow \hat{\mathbf{A}}^T \odot \hat{\mathbf{u}} \quad \hat{\mathbf{y}}[i] = \sum_{j=0}^{k-1} \hat{\mathbf{A}}[j, i] \odot \hat{\mathbf{u}}[j] \quad (2.13)$$

$$\hat{z} \leftarrow \hat{\mathbf{u}}^T \odot \hat{\mathbf{v}} \quad \hat{z} = \sum_{j=0}^{k-1} \hat{\mathbf{u}}[j] \odot \hat{\mathbf{v}}[j] \quad (2.14)$$

2.2.7 Applying Algorithms to Arrays, Examples

In the previous examples, arithmetic over $\mathbb{Z}_m$ was extended to arithmetic with arrays over $\mathbb{Z}_m$ and then further extended to arithmetic with matrices whose entries are themselves arrays over $\mathbb{Z}_m$. Similarly, algorithms defined with a given data type as input will be applied to arrays and matrices over that data type. When the pseudocode invokes such an algorithm on an array or matrix input, it is implied that the algorithm is invoked repeatedly and applied to each entry of the input.

For example, consider the function $\text{Comp}_d : \mathbb{Z}_q \rightarrow \mathbb{Z}_p$ (defined in Section 2.3.3), where $p = (q - 1)/2^d$. It can be invoked on an array input $F \in \mathbb{Z}_q^n$ with the statement

$$K \leftarrow \text{Comp}_d(F). \quad (2.15)$$

The result will be an array $K \in \mathbb{Z}_p^n$ such that $K[i] = \text{Comp}_d(F[i])$ for every $i \in \{0, 1, \dots, n-1\}$. The computation (2.15) involves running the Comp_d algorithm n times.

For a second example, consider the algorithm **NTT** defined. It takes an array $f \in \mathbb{Z}_q^n$ (representing an element of R_q) as input and outputs another array $\hat{f} \in \mathbb{Z}_q^n$ (representing an element of T_q). If the **NTT** algorithm is invoked on a vector $\mathbf{s} \in (\mathbb{Z}_q^n)^k$ (representing an element of R_q^k) with the pseudocode statement

$$\hat{\mathbf{s}} \leftarrow \text{NTT}(\mathbf{s}), \quad (2.16)$$

the result is a vector $\hat{\mathbf{s}} \in (\mathbb{Z}_q^n)^k$ such that $\hat{\mathbf{s}}[i] = \text{NTT}(\mathbf{s}[i])$ for all $i \in \{0, 1, \dots, k-1\}$. The vector $\hat{\mathbf{s}}$ represents an element of T_q^k . The computation (2.16) involves running the **NTT** algorithm k times.

2.3 Basic Operations

2.3.1 Hashing

Expanding the Matrix $\mathbf{A}$. The function `ExpandA` maps a uniform seed $\rho \in \{0, 1\}^n$ to a matrix $\mathbf{A} \in R_q^{k \times l}$ in NTT domain representation. The matrix $\mathbf{A}$ is only needed for multiplication. Hence, for the sake of faster implementations, the expansion function `ExpandA` does not output $\mathbf{A} \in R_q^{k \times l} = (\mathbb{Z}_q[X]/(X^n+1))^{k \times l}$. Instead it outputs $\hat{\mathbf{A}} \in \mathbb{Z}_q^{k \times l \times n}$, which is interpreted as the NTT domain representation of $\mathbf{A}$. As $\mathbf{A}$ needs to be sampled uniformly and the NTT is an isomorphism, `ExpandA` also needs to sample uniformly in this representation. To be compatible to COMPASS-SIG, an implementation whose NTT produces differently ordered vectors than our reference NTT needs to sample coefficients in a non-consecutive order.

Sampling the vectors $\mathbf{s}$. The function `Sample`, used for generating the secret vectors in key generation, given a distribution β_{η_s} , maps a seed σ to $\mathbf{s} \in \beta_{\eta_s}^\ell$.

Hashing. The documentation will also make use of three hash functions $\mathbf{H}$, $\mathbf{J}$ and $\mathbf{G}$, which are defined as follows.

The functions $\mathbf{H}$ and $\mathbf{J}$ each take one variable-length input and produce one n -bit ($n/8$ -byte) output. They will be denoted by $\mathbf{H} : \mathbb{B}^* \rightarrow \mathbb{B}^{n/8}$ and $\mathbf{J} : \mathbb{B}^* \rightarrow \mathbb{B}^{n/8}$, respectively. They can be instantiated from a secure hash standard with different operation modes or different prefixes.

The function $\mathbf{G}$ takes one variable-length input and produces two n -bit ($n/8$ -byte) outputs. It will be denoted by $\mathbf{G} : \mathbb{B}^* \rightarrow \mathbb{B}^{n/8} \times \mathbb{B}^{n/8}$. The two outputs of $\mathbf{G}$ will be denoted by $(a, b) \leftarrow \mathbf{G}(c)$, where $a, b \in \mathbb{B}^{n/8}$, $c \in \mathbb{B}^*$, and $\mathbf{G}(c) = a \| b$. The function can be instantiated from a secure hash standard with operation mode or prefix different from the previous functions.

2.3.2 NTT domain representation

The Number-Theoretic Transform (NTT) can be viewed as a specialized, exact version of the discrete Fourier transform. In the case of s COMPASS-KEM, the NTT is used to improve the efficiency of multiplication in the ring R_q . Recall that R_q is the ring $\mathbb{Z}_q[X]/(X^n+1)$ of polynomials of the form $f = f_0 + f_1X + \dots + f_{n-1}X^{n-1}$ (where $f_j \in \mathbb{Z}_q$ for all j), with the ring operations defined by arithmetic modulo $X^n + 1$.

The ring R_q is isomorphic to another ring T_q , which is a direct sum of $n/2$ quadratic extensions of $\mathbb{Z}_q$. The NTT is a computationally efficient isomorphism between these two rings. When a polynomial $f \in R_q$ is input, the NTT outputs an element $\hat{f} := \text{NTT}(f)$ of the ring T_q , where $\hat{f}$ is called the *NTT representation* of f . The isomorphism property implies that

$$f \cdot g = \text{INTT}(\hat{f} \odot \hat{g}), \quad (2.17)$$

where $\cdot$ and $\odot$ denote multiplication in R_q and T_q , respectively. Moreover, since T_q is a product of $n/2$ rings that each consist of polynomials of degree at most one, the operation $\odot$ is much more efficient than the operation $\cdot$. For these reasons, the NTT is considered to be an integral part of s COMPASS-KEM and not merely an optimization.

As the rings R_q and T_q have a vector space structure over $\mathbb{Z}_q$, the most natural abstract data type to represent elements of either of these rings is $\mathbb{Z}_q^n$. For this reason, the choice

of data structure for the inputs and outputs of NTT and INTT are length- n arrays of integers modulo q (see Section 2.2.4). However, to clarify the distinction between the algebraic objects before and after the conversion, the algorithms are written with explicit inputs and outputs. This is consistent with this standard's convention that all inputs are passed by copy.

The mathematical structure of the NTT. For both parameter sets, there exist primitive n -th roots of unity but no primitive $2n$ -th roots of unity in $\mathbb{Z}_q$. We take ζ to be a primitive n -th root of unity in $\mathbb{Z}_q$.

Define $\text{BitRev}_{\log n-1}(i)$ to be the integer represented by bit-reversing the unsigned $\log n - 1$ -bit value corresponding to the input integer $i \in \{0, \dots, n/2 - 1\}$.

The polynomial $X^n + 1$ factors into $n/2$ polynomials of degree 2 modulo q as follows:

$$X^n + 1 = \prod_{i=0}^{n/2-1} (X^2 - \zeta^{2\text{BitRev}_{\log n-1}(i)+1}). \quad (2.18)$$

Therefore, $R_q := \mathbb{Z}_q[X]/(X^n + 1)$ is isomorphic to a direct sum of $n/2$ quadratic extensions of $\mathbb{Z}_q$, denoted T_q . Specifically, this ring has the structure

$$T_q = \bigoplus_{i=0}^{n/2-1} \mathbb{Z}_q[X]/(X^2 - \zeta^{2\text{BitRev}_{\log n-1}(i)+1}). \quad (2.19)$$

Thus, the NTT representation $\hat{f} \in T_q$ of a polynomial $f \in R_q$ is the vector that consists of the corresponding residues of degree at most one:

$$\hat{f} := (f \bmod (X^2 - \zeta^{2\text{BitRev}_{\log n-1}(0)+1}), \dots, f \bmod (X^2 - \zeta^{2\text{BitRev}_{\log n-1}(n/2-1)+1})). \quad (2.20)$$

As discussed in Section 2.2.4, the algorithms in this standard represent $\hat{f}$ as an array of n integers modulo q . Specifically,

$$f \bmod (X^2 - \zeta^{2\text{BitRev}_{\log n-1}(i)+1}) = \hat{f}[2i] + \hat{f}[2i+1]X, \quad (2.21)$$

for i from 0 to $n/2 - 1$.

The multiplication in T_q naturally follows the definition below: for $\hat{f}, \hat{g} \in T_q$, $\hat{h} = \hat{f} \odot \hat{g} \in T_q$ consists of elements $\hat{h}[0], \dots, \hat{h}[n-1]$, where:

$$\hat{h}[2i] + \hat{h}[2i+1]X = (\hat{f}[2i] + \hat{f}[2i+1]X)(\hat{g}[2i] + \hat{g}[2i+1]X) \bmod (X^2 - \zeta^{2\text{BitRev}_{\log n-1}(i)+1}) \quad (2.22)$$

for i from 0 to $n/2 - 1$.

2.3.3 Compression and Decompression

Compression Function $\text{Comp}_d(x)$ For $x \in \mathbb{Z}_q$, the compression operation with parameter d is performed as follows:

1. If $x > 0$, set $x \leftarrow x - 1$;
2. Return $\lfloor x/2^d \rfloor$.

For $x \in R_q^k$, Comp_d is applied to each coefficient of the elements in x .

It is not hard to see that, if $2^d \mid q-1$, $\text{Comp}_d(x) = \lfloor \frac{p}{q} \cdot x \rfloor$ for $p = (q-1)/2^d$, which fits the definition of rounding operation in the hardness assumption MLWR (Section 3.1).

Decompression Function $\text{Decomp}_d(x)$ For $x \in \mathbb{Z}_q$, the decompression operation with parameter d returns $x \cdot 2^d + 2^{d-1}$. For $x \in R_q^k$, Decomp_d is applied to each coefficient of the elements in x . The following condition is satisfied: $|x - \text{Decomp}_d(\text{Comp}_d(x))| \leq 2^{d-1}$.

We should note that both Comp_d and Decomp_d cannot be applied to the NTT domain.

2.4 COMPASS-PKE Algorithms

This section describes the component scheme COMPASS-PKE. COMPASS-PKE is not approved for use in a stand-alone fashion. It serves only as a collection of subroutines for use in the algorithms of the approved scheme COMPASS-KEM.

COMPASS-PKE consists of three algorithms:

1. Key generation (COMPASS-PKE.KeyGen).
2. Encryption (COMPASS-PKE.Encrypt).
3. Decryption (COMPASS-PKE.Decrypt).

When COMPASS-PKE is instantiated as part of COMPASS-KEM, COMPASS-PKE inherits the parameter set selected for COMPASS-KEM. Each parameter set specifies numerical values for each parameter. Parameters and parameter sets are described in Section 3.2.

The algorithms in this section do not perform any input checking because they are only invoked as subroutines of the main COMPASS-KEM algorithms. The algorithms of COMPASS-KEM themselves do perform input checking as needed.

Each of the algorithms of COMPASS-PKE is accompanied by a brief, informal description in text. For simplicity, this description is written in terms of vectors and matrices whose entries are elements of R_q . In the actual algorithm, most of the computations occur in the NTT domain in order to improve the efficiency of multiplication. The relevant vectors and matrices will then have entries in T_q . Linear-algebraic arithmetic with such vectors and matrices is performed as described .

2.4.1 Key Generation (KeyGen)

The key generation algorithm COMPASS-PKE.KeyGen outputs an encryption key pk and a decryption key sk . As is typically the case for public-key encryption, the encryption key can be made public, while the decryption key and the randomness must remain private. Indeed, the encryption key of COMPASS-PKE will serve as the encapsulation key of COMPASS-KEM (see COMPASS-KEM.KeyGen below) and can thus be made public. Meanwhile, the decryption key and seed of COMPASS-PKE.KeyGen must remain private as they can be used to perform decapsulation in COMPASS-KEM.

The matrix $\hat{\mathbf{A}}$ generated in step 3 of COMPASS-PKE.KeyGen can be stored, as specified. This allows later operations to use $\hat{\mathbf{A}}$ directly rather than re-expanding it from the public seed ρ .

The decryption key of COMPASS-PKE.KeyGen is a length- k vector s of elements of R_q (i.e., $s \in R_q^k$). Roughly speaking, s is a set of secret variables, while the encryption key is a collection of compressed linear equations $(\mathbf{A}, \text{Comp}_d(\mathbf{A}s))$ in the secret variables s . The rows of the matrix $\mathbf{A}$ form the equation coefficients. This matrix is generated pseudorandomly using XOF. Once $\mathbf{A}$ and s are generated, the computation of the final

part $\mathbf{t} = \text{Comp}_d(\mathbf{A}\mathbf{s})$ of the encryption key takes place. The results are appropriately encoded into byte arrays and output.

In $\text{COMPASS-PKE.KeyGen}$, the choice of parameter set affects the length of the secret s (via the parameter k) and, as a consequence, the sizes of the pseudorandom matrix $\mathbf{A}$. The choice of parameter set also affects the distribution (via the parameter η_s) used to sample the entries of $\mathbf{s}$.

Algorithm 1 $\text{COMPASS-PKE.KeyGen}()$

Input: None

1. $\iota \xleftarrow{\$} \mathbb{B}^{n/8}$ ▷ Randomly sample n -bit initial seed
2. $(\rho, \sigma) \leftarrow \mathbf{G}(\iota)$ ▷ Derive ρ and σ via hash function G
3. $\hat{\mathbf{A}} \leftarrow \text{ExpandA}(\rho) \in R_q^{k \times l}$ ▷ Construct matrix in NTT domain using ρ
4. $\mathbf{s} \leftarrow \text{Sample}(\beta_{\eta_s}, \sigma) \in R^l$ ▷ Sample private vector $\mathbf{s}$ using σ
5. $\hat{\mathbf{s}} \leftarrow \text{NTT}(\mathbf{s})$ ▷ Transform private key to NTT domain
6. $\mathbf{t}' \leftarrow \text{INTT}(\hat{\mathbf{A}} \odot \hat{\mathbf{s}})$ ▷ Multiply in NTT domain and transform back
7. $\mathbf{t} \leftarrow \text{Comp}_d(\mathbf{t}')$ ▷ Compress public key
8. $pk \leftarrow (\rho, \mathbf{t})$
9. $sk \leftarrow (\iota, \hat{\mathbf{s}})$

Output: Public/Private key pair (pk, sk)

2.4.2 Encryption (Encrypt)

The encryption algorithm $\text{COMPASS-PKE.Encrypt}$ of COMPASS-PKE takes an encryption key ek_{PKE} , a $n/8$ -byte plaintext $\mathbf{m}$, and randomness $\mathbf{r}$ as input and produces a single output ciphertext c .

The matrix $\hat{\mathbf{A}}$ generated in steps 1 of $\text{COMPASS-PKE.Encrypt}$ can be stored, as specified. This allows later operations to use $\hat{\mathbf{A}}$ directly rather than re-expanding it from the public seed ρ .

The algorithm $\text{COMPASS-PKE.Encrypt}$ begins by extracting the vector $\mathbf{t}$ and the seed from the encryption key. The seed is then expanded to re-generate the matrix $\mathbf{A}$ in the same manner as was done in $\text{COMPASS-PKE.KeyGen}$. If $\mathbf{t}$ and $\mathbf{A}$ are derived correctly from an encryption key produced by $\text{COMPASS-PKE.KeyGen}$, then they are equal to their corresponding values in $\text{COMPASS-PKE.KeyGen}$.

Recall from the description of key generation that the pair $(\mathbf{A}, \text{Decomp}_d(\mathbf{t}))$, where $\mathbf{t} = \text{Comp}_d(\mathbf{A}\mathbf{s})$ can be thought of as a system of compressed linear equations in the secret variables $\mathbf{s}$. One can generate an additional noisy linear equation in the same secret variables — without knowing $\mathbf{s}$ — by picking a random linear combination of the noisy equations in the system $(\mathbf{A}, \text{Decomp}_d(\mathbf{t}))$. One can then encode information in the “constant term” (i.e., the entry that is a linear combination of entries of $\mathbf{t}$) of such a combined equation. This information can then be deciphered by a party in possession of $\mathbf{s}$. For example, one could encode a single bit by deciding whether or not to significantly alter the constant term, thus making either a nearly correct equation that corresponds to the decrypted bit value of 0 or a far-from-correct equation that corresponds to the decrypted bit value of 1. In the case of COMPASS-PKE , the constant term is a polynomial with n coefficients, so one can encode more information: one bit in each coefficient.

To this end, $\text{COMPASS-PKE.Encrypt}$ proceeds by generating a vector $\mathbf{r} \in R_q^k$. One then computes the “new compressed equation”, which is $(\text{Comp}_{d_1}(\mathbf{A}^T \mathbf{r}), \text{Comp}_{d_2}(\mathbf{t}^T \mathbf{r}))$ (up to some details). An appropriate encoding of the input message m is then added to the

latter term in the pair. Finally, the resulting pair (c_1, c_2) is serialized into a byte array, and output as the ciphertext.

Algorithm 2 COMPASS-PKE.Encrypt(pk, m, r)

Input: $pk = (\rho, \mathbf{t})$; n -bit message $m \in \{0, 1\}^n$; random seed $r \in \mathbb{B}^{n/8}$

1. $\hat{\mathbf{A}} \leftarrow \text{ExpandA}(\rho) \in R_q^{k \times l}$ ▷ Recover $\hat{\mathbf{A}}$ from $pk.seed$
2. $\mathbf{t}' \leftarrow \text{Decomp}_d(\mathbf{t})$ ▷ Decompress public key
3. $\mathbf{r} \leftarrow \text{Sample}(\beta_{\eta_2}, r) \in R^k$ ▷ Sample $n \times k$ elements from β_{η_2}
4. $\hat{\mathbf{r}} \leftarrow \text{NTT}(\mathbf{r})$
5. $\mathbf{c}'_1 \leftarrow \text{INTT}(\hat{\mathbf{A}}^T \odot \hat{\mathbf{r}})$
6. $\mathbf{c}'_2 \leftarrow \text{INTT}(\text{NTT}(\mathbf{t}')^T \odot \hat{\mathbf{r}})$
7. $\mathbf{c}'_2 \leftarrow \mathbf{c}'_2 + m \cdot \lfloor q/2 \rfloor \pmod{q}$ ▷ Encode message into high bits
8. $\mathbf{c}_1 \leftarrow \text{Comp}_{d_1}(\mathbf{c}'_1)$
9. $\mathbf{c}_2 \leftarrow \text{Comp}_{d_2}(\mathbf{c}'_2)$

Output: $c = (c_1, c_2)$

2.4.3 Decryption (Decrypt)

The decryption algorithm COMPASS-PKE.Decrypt of COMPASS-PKE takes a decryption key sk and a ciphertext c as input, requires no randomness, and outputs a plaintext m . The algorithm COMPASS-PKE.Decrypt begins by recovering a pair $(\mathbf{c}'_1, \mathbf{c}'_2)$ from the ciphertext c (see the description of COMPASS-PKE.Encrypt). Here, one can think of $\mathbf{c}'_1$ as the coefficients of the equation and $\mathbf{c}'_2$ as the constant term. The decryption key sk contains the vector of secret variables $\mathbf{s}$. The decryption algorithm can thus use the decryption key to compute the true constant term $v = \mathbf{s}^T \mathbf{c}'_1$ and calculate $\mathbf{c}'_2 - v$. The decryption algorithm ends by decoding the plaintext message m from $\mathbf{c}'_2 - v$ and outputting m .

Algorithm 3 COMPASS-PKE.Decrypt(sk, ct)

Input: $sk = (z, \hat{\mathbf{s}})$; Ciphertext $ct = (\mathbf{c}_1, \mathbf{c}_2)$

1. $\mathbf{c}'_1 \leftarrow \text{Decomp}_{d_1}(\mathbf{c}_1)$
2. $\mathbf{c}'_2 \leftarrow \text{Decomp}_{d_2}(\mathbf{c}_2)$
3. $\mathbf{u} \leftarrow \mathbf{c}'_2 - \text{INTT}(\text{NTT}(\mathbf{c}'_1)^T \odot \hat{\mathbf{s}}) \pmod{q}$
4. **for** $i \leftarrow 0$ **to** $n - 1$ **do**
5. **if** $\lfloor q/4 \rfloor \leq \mathbf{u}[i] < \lfloor 3q/4 \rfloor$ **then**
6. $m[i] \leftarrow 1$
7. **else**
8. $m[i] \leftarrow 0$
9. **end if**
10. **end for**

Output: Message $m \in \{0, 1\}^n$

2.5 Main Algorithms

This section specifies three algorithms: COMPASS-KEM.KeyGen, COMPASS-KEM.Encaps, and COMPASS-KEM.Decaps. The algorithms in this section make use of the parameters $n, q, k \times l, (d, d_1, d_2)$. The subroutines they invoke additionally make use of the parameters η_s and η_2 .

2.5.1 Key Generation

The algorithm `COMPASS-KEM.KeyGen` produces an encapsulation key and a decapsulation key.

The core subroutine of `COMPASS-KEM.KeyGen` is the key generation algorithm of `COMPASS-PKE`. The encapsulation key is simply the encryption key of `COMPASS-PKE`. The decapsulation key consists of the decryption key of `COMPASS-PKE`, the encapsulation key, a hash of the encapsulation key, and a random $n/8$ -byte value. This random value will be used in the “implicit rejection” mechanism of the decapsulation algorithm.

Algorithm 4 `COMPASS-KEM.KeyGen()`

Input: None

1. $(\mathbf{ek}_{\text{PKE}}, \mathbf{dk}_{\text{PKE}}) \leftarrow \text{COMPASS-PKE.KeyGen}()$ ▷
 Run key generation for `COMPASS-PKE`
2. $z \xleftarrow{\$} \mathbb{B}^{n/8}$
3. $\mathbf{ek} \leftarrow \mathbf{ek}_{\text{PKE}}$ ▷ KEM encaps key is just the PKE encryption key
4. $\mathbf{dk} \leftarrow (\mathbf{dk}_{\text{PKE}} \parallel \mathbf{ek} \parallel \mathbf{H}(\mathbf{ek}) \parallel z)$ ▷ KEM decaps key includes PKE decryption key
5. **return** $(\mathbf{ek}, \mathbf{dk})$

Output: Encapsulation key $\mathbf{ek}$, decapsulation key $\mathbf{dk}$

2.5.2 Encapsulation

The algorithm `COMPASS-KEM.Encaps` accepts an encapsulation key as input and outputs a ciphertext and a shared key.

The core subroutine of `COMPASS-KEM.Encaps` is the encryption algorithm of `COMPASS-PKE`, which is used to encrypt a random value m into a ciphertext c . A copy of the shared secret key K and the randomness used during encryption are derived from m and the encapsulation key $\mathbf{ek}$ via hashing. Specifically, $\mathbf{H}$ is applied to $\mathbf{ek}$, and the result is concatenated with m and then hashed using $\mathbf{G}$. Finally, the algorithm outputs the shared secret key K and the ciphertext c .

Algorithm 5 `COMPASS-KEM.Encaps(ek)`

Input: Encapsulation key $\mathbf{ek}$

1. $m \xleftarrow{\$} \mathbb{B}^{n/8}$
2. $(K, r) \leftarrow \mathbf{G}(m \parallel \mathbf{H}(\mathbf{ek}))$ ▷ Derive shared secret key K and randomness r
3. $c \leftarrow \text{COMPASS-PKE.Encrypt}(\mathbf{ek}, m, r)$ ▷
 Encrypt m using `COMPASS-PKE` with randomness r
4. **return** (K, c)

Output: Shared secret key $K \in \mathbb{B}^{n/8}$, ciphertext c

2.5.3 Decapsulation

The algorithm `COMPASS-KEM.Decaps` accepts a decapsulation key and a ciphertext as input, does not use any randomness, and outputs a shared secret key.

The algorithm `COMPASS-KEM.Decaps` begins by parsing out the components of the decapsulation key $\mathbf{dk}$ of `COMPASS-KEM`. These components are an (encryption key, decryption key) pair for `COMPASS-PKE`, a hash value h , and a random value z . The decryption key of `COMPASS-PKE` is then used to decrypt the input ciphertext c to get a

plaintext m' . The decapsulation algorithm then re-encrypts m' and computes a candidate shared secret key K' in the same manner as should have been done in encapsulation. Specifically, K' and the encryption randomness r' are computed by hashing m' and the encryption key of COMPASS-PKE, and a ciphertext c' is generated by encrypting m' using randomness r' . Finally, decapsulation checks whether the resulting ciphertext c' matches the provided ciphertext c . If it does not, the algorithm performs an “implicit rejection”: the value of K' is changed to a hash of c together with the random value z stored in the COMPASS-KEM secret key. In either case, decapsulation outputs the resulting shared secret key K' .

The “implicit reject” flag computed in step 6 (by comparing c and c') is a secret piece of intermediate data. This flag shall be destroyed prior to COMPASS-KEM.Decaps terminating. In particular, returning the value of the flag as an output in any form is not permitted.

Algorithm 6 COMPASS-KEM.Decaps(dk, c)

Input: Decapsulation key dk, ciphertext c

1. $(\text{dk}_{\text{PKE}}, \text{ek}_{\text{PKE}}, h, z) \leftarrow \text{dk}$ ▷ Extract all opponents from the decapsulation key
2. $m' \leftarrow \text{COMPASS-PKE.Decrypt}(\text{dk}_{\text{PKE}}, c)$ ▷ Decrypt ciphertext
3. $(K', r') \leftarrow \text{G}(m' || h)$ ▷ Derive shared secret and randomness
4. $\bar{K} \leftarrow \text{J}(z || c)$ ▷ Compute implicit rejection key
5. $c' \leftarrow \text{COMPASS-PKE.Encrypt}(\text{ek}_{\text{PKE}}, m', r')$ ▷ Re-encrypt to verify c
6. **if** $c \neq c'$ **then**
7. $K' \leftarrow \bar{K}$ ▷ Implicitly reject if ciphertexts mismatch
8. **end if**
9. **return** K'

Output: Shared secret key $K \in \mathbb{B}^{n/8}$

3. Design Rationale

3.1 Security Foundation

The cryptographic constructions presented in this document are based on the plain Module Learning With Rounding (MLWR) problem and a new hardness problem called biased MLWR. We first define the rounding operation: $\lfloor x \rfloor_p$ for $x \in \mathbb{Z}_q$ is defined as $\lfloor \frac{p}{q} \cdot x \rfloor$. For $2^d \mid q - 1$, we can see that $\text{Comp}_d(x) = \lfloor x \rfloor_p$ for $p = (q - 1)/2^d$.

Definition 1. (MLWR[DKRV18]) Let $q \geq p \geq 2$, public term $\mathbf{A} \xleftarrow{\$} R_q^{k \times l}$ and secret term $\mathbf{s} \xleftarrow{\$} R_q^l$. Then the $\text{MLWR}_{n,q,p,k,l}$ distribution over $R_q^{k \times l} \times R_p^k$ is defined as

$$(\mathbf{A}, \mathbf{b} = \lfloor \mathbf{A}\mathbf{s} \rfloor_p).$$

The (decision) $\text{MLWR}_{n,q,p,k,l}$ problem is defined as distinguishing the $\text{MLWR}_{n,q,p,k,l}$ distribution from $(\mathbf{A}, \lfloor \mathbf{u} \rfloor_p)$, where $(\mathbf{A}, \mathbf{u})$ is a uniformly random distribution over $R_q^{k \times l} \times R_q^k$.

Definition 2. (Biased MLWR) Let $q \geq p \geq 2$, public term $\tilde{\mathbf{A}} \xleftarrow{\$} R_q^{k \times l}, \mathbf{t} \xleftarrow{\$} R_q^k$ and secret term $\mathbf{s} \xleftarrow{\$} R_q^k$. Set $\mathbf{t}' = \text{Decomp}_d(\text{Comp}_d(\mathbf{t}))$, $\mathbf{A} = [\tilde{\mathbf{A}} | \mathbf{t}']$. Then the biased $\text{MLWR}_{n,q,p,d,k,l}$ distribution over $R_q^{k \times (l+1)} \times R_p^{l+1}$ is defined as

$$(\mathbf{A}, \mathbf{b} = \lfloor \mathbf{s}^T \mathbf{A} \rfloor_p).$$

The (decision) biased $\mathbf{MLWR}_{n,q,p,d,k,l}$ problem is defined as distinguishing the biased $\mathbf{MLWR}_{n,q,p,d,k,l}$ distribution from the distribution $(\mathbf{A}, [\mathbf{u}]_p)$ over $R_q^{k \times (l+1)} \times R_p^{l+1}$, where $\mathbf{u} \xleftarrow{\$} R_q^{l+1}$.

It is necessary to point out that SABER [DKRV18], the only MLWR-based scheme with a formal proof has its security reduction flawed. To fix such an issue, it is inevitable to introduce biased MLWR as a new hard problem in the security proof. The new problem does not affect the concrete security estimation, as we shall show below.

The hardness of plain MLWR can be reduced to the hardness of MLWE (or RLWE) by the work [LW20], yet the theoretical hardness of biased MLWR is not straightforward. In this part, we present a reduction from MLWE to biased MLWR, to show that introducing the non-standard variant of MLWR does not lower the provable security of our scheme.

Theorem 1. *Let $q \geq p \geq 2$. There exists a poly-time reductions from $\mathbf{MLWE}$ to biased $\mathbf{MLWR}$ defined above.*

Proof. The target is to show that the element $\mathbf{b} = [\mathbf{s}^T \mathbf{A}]_p$ in the biased $\mathbf{MLWR}$ sample is indistinguishable from the uniformly random element.

Firstly, the public term $\mathbf{A}$ can be rewritten as $\mathbf{A} = [\tilde{\mathbf{A}}|\mathbf{t} + \mathbf{z}]$, where $\|\mathbf{z}\|_\infty \leq 2^{d-1}$, according to the definitions of the compression and decompression operations. Then the element $\mathbf{b} = [\mathbf{s}^T \mathbf{A}]_p = [[\mathbf{s}^T \tilde{\mathbf{A}}|\mathbf{s}^T \mathbf{t} + \mathbf{s}^T \mathbf{z}]]_p$.

Second, the uniformly random elements $\tilde{\mathbf{A}}$ and $\mathbf{t}$ can be replaced by $\mathbf{CB} + \mathbf{F}$ and $\mathbf{c}^T \mathbf{B} + \mathbf{f}$ by $\mathbf{MLWE}$ assumption, where $\mathbf{C}, \mathbf{c}$ and $\mathbf{F}, \mathbf{f}$ are with small norm. As a result, the element $\mathbf{b}$ can be further rewritten as $\mathbf{b} = [[\mathbf{s}^T \mathbf{CB} + \mathbf{s}^T \mathbf{F} | (\mathbf{cs})^T \mathbf{B} + \mathbf{s}^T \mathbf{f} + \mathbf{s}^T \mathbf{z}]]_p$.

Next, we need prove that the remaining min-entropy of $\mathbf{s}$ is still sufficient for random extraction, i.e.,

$$H_\infty \left(\mathbf{s} | \bar{\mathbf{A}}, [[\mathbf{s}^T \mathbf{CB} + \mathbf{s}^T \mathbf{F} | (\mathbf{cs})^T \mathbf{B} + \mathbf{s}^T \mathbf{f} + \mathbf{s}^T \mathbf{z}]]_p \right)$$

is sufficiently large, where we set $\bar{\mathbf{A}} = [\mathbf{CB} + \mathbf{F} | \mathbf{c}^T \mathbf{B} + \mathbf{f} + \mathbf{z}]$ for simplicity. To this end, we split the secret $\mathbf{s}$ into two parts: $\mathbf{s} = \mathbf{s}_1 + \mathbf{s}_2$, where $\mathbf{s}_1$ is uniformly random and $\mathbf{s}_1$ is γ bounded, and define the set I as the indexes and the corresponding coefficients of $\mathbf{s}^T \bar{\mathbf{A}}$ such that $[\mathbf{s}^T \bar{\mathbf{A}}]_p \neq [[\mathbf{s}^T \mathbf{CB} | (\mathbf{cs})^T \mathbf{B}]]_p$. Then by a similar argument as [LW20] (argument based on Chernoff bounds), the size of I is bounded. Now, $[\mathbf{s}^T \bar{\mathbf{A}}]_p$ can be reconstructed by the terms $\mathbf{s}^T \mathbf{C}, \mathbf{C}, \mathbf{B}, \mathbf{cs}^T, \mathbf{c}$ and I . Therefore

$$\begin{aligned} H_\infty (\mathbf{s} | \bar{\mathbf{A}}, [\mathbf{s}^T \bar{\mathbf{A}}]_p) &= H_\infty \left(\mathbf{s} | \bar{\mathbf{A}}, [[\mathbf{s}^T \mathbf{CB} + \mathbf{s}^T \mathbf{F} | (\mathbf{cs})^T \mathbf{B} + \mathbf{s}^T \mathbf{f} + \mathbf{s}^T \mathbf{z}]]_p \right) \\ &\geq H_\infty (\mathbf{s} | \mathbf{s}_2^T \mathbf{C}, \mathbf{C}, \mathbf{B}, (\mathbf{cs}_2)^T, \mathbf{c}, \mathbf{s}_1 \bar{\mathbf{A}}, I) \\ &\geq H_\infty (\mathbf{s} | \mathbf{s}_1) - (n + \lambda) \log q \\ &\geq kn \log(2\gamma) - (n + \lambda) \log q. \end{aligned}$$

By setting the parameters appropriately, i.e., $kn \log(2\gamma) - (n + \lambda) \log q \geq n \log(q) + 2\lambda$, we can apply the ring-based leftover hash lemma (ref. [LW20]) to prove that

$$\left(\begin{bmatrix} [\bar{\mathbf{A}}]_p \\ [\mathbf{a}]_p \end{bmatrix}, \begin{bmatrix} [\mathbf{s}^T \bar{\mathbf{A}}]_p \\ [\langle \mathbf{a}, \mathbf{s} \rangle]_p \end{bmatrix} \right) \stackrel{s}{\approx} \left(\begin{bmatrix} [\bar{\mathbf{A}}]_p \\ [\mathbf{a}]_p \end{bmatrix}, \begin{bmatrix} [\mathbf{s}^T \bar{\mathbf{A}}]_p \\ [u]_p \end{bmatrix} \right).$$

The exact statistical distance is bounded by $2^{-\lambda}$. Finally, we replace the $\bar{\mathbf{A}}$ with $\mathbf{A}$ by $\mathbf{MLWE}$ to get:

$$\left(\begin{bmatrix} [\bar{\mathbf{A}}]_p \\ [\mathbf{a}]_p \end{bmatrix}, \begin{bmatrix} [\mathbf{s}^T \bar{\mathbf{A}}]_p \\ [u]_p \end{bmatrix} \right) \stackrel{c}{\approx} \left(\begin{bmatrix} [\mathbf{A}]_p \\ [\mathbf{a}]_p \end{bmatrix}, \begin{bmatrix} [\mathbf{s}^T \mathbf{A}]_p \\ [u]_p \end{bmatrix} \right).$$

By a simple hybrid argument, we can further prove the desired statement

$$(\mathbf{A}, \lfloor \mathbf{s}^T \mathbf{A} \rfloor_p) \stackrel{c}{\approx} (\mathbf{A}, \lfloor \mathbf{u} \rfloor_p).$$

□

3.2 Parameter Choices

To accommodate different security requirements, we provide four parameter sets targeting 128, 256, 384, and 512 bits of security, respectively. All parameter sets use a prime modulus q , a ring degree n of either 256 or 512, and appropriate matrix dimensions $(k \times l)$. For each set, the coefficients of the secret vector $\mathbf{s}$ and encryption randomness $\mathbf{r}$ are taken from centered binomial distributions with parameters $\{2, 3, 4\}$ (i.e., $\eta_s = \eta_2 \in \{2, 3, 4\}$), which ensure the security level of the scheme as well as the decryption correctness. The parameter d, d_1, d_2 defines the number of bits used for compression and decompression, reducing the public key and ciphertext sizes, where $d = d_1 = 2$ and $d_2 \in \{6, 8\}$ for different security levels. All parameters have been carefully chosen to guarantee moderate public key and ciphertext sizes, decryption correctness, and resistance against known lattice attacks (e.g., primal and dual attacks). The exact numerical values are listed in the following table.

Table 1: Mapping of Security Levels to COMPASS-KEM Parameters							
Level	n	q	p	$k \times l$	η_s	(d, d_1, d_2)	Required Security
Level 1	256	3329	832	2×2	3	(2, 2, 8)	128 bit
Level 2	256	3329	832	4×4	2	(2, 2, 6)	256 bit
Level 3	512	7681	1920	3×3	3	(2, 2, 8)	384 bit
Level 4	512	7681	1920	4×4	4	(2, 2, 6)	512 bit

Below we give the key and ciphertext sizes of COMPASS-KEM under different security levels, and the comparison to ML-KEM under similar security. We can see that our scheme outperforms ML-KEM in key and ciphertext sizes under both 128 and 256 bit security levels.

Table 2: Key and Ciphertext Sizes for COMPASS-KEM (bytes)				
Algorithm Name	Security Level	Public Key	Private Key	Ciphertext
COMPASS-KEM-128	128 bit	672	1504	768
ML-KEM-512	128 bit	800	1632	768
COMPASS-KEM-256	256 bit	1312	2912	1472
ML-KEM-1024	256 bit	1568	3168	1568
COMPASS-KEM-384	384 bit	2144	4704	2432
COMPASS-KEM-512	512 bit	2848	6240	3264

3.3 Decryption Failure Analysis

Here we analysis the decryption failure rate of our COMPASS-KEM scheme. It is hold that our COMPASS-KEM scheme and subroutine COMPASS-PKE share the same failure

rate. In the context of decryption algorithm of COMPASS-PKE, the following equations are satisfied modulo q :

$$\begin{aligned} u &= c'_2 - \mathbf{c}'_1 \cdot \mathbf{s} = m \cdot \frac{q-1}{2} + (\mathbf{A} \cdot \mathbf{s} + \mathbf{e}_t)^T \cdot \mathbf{r} + e_2 - (\mathbf{A}^T \cdot \mathbf{r} + \mathbf{e}_1)^T \cdot \mathbf{s} \\ &= m \cdot \frac{q-1}{2} + \mathbf{e}_t^T \cdot \mathbf{r} + e_2 - \mathbf{e}_1^T \cdot \mathbf{s}, \end{aligned}$$

where $\mathbf{e}_t$, $\mathbf{e}_1$ and e_2 are errors caused by compression. Therefore, the decryption is correct if the infinity norm of $e_{\text{total}} := \mathbf{e}_t^T \cdot \mathbf{r} + e_2 - \mathbf{e}_1^T \cdot \mathbf{s}$ is less than $\frac{q-1}{4}$. To evaluate the decryption failure rate, we model that $\mathbf{s} \leftarrow \beta_{\eta_s}$, $\mathbf{r} \leftarrow \beta_{\eta_2}$, $\mathbf{e}_t \leftarrow \{\text{Decomp}_d(\text{Comp}_d(x)) - x \mid x \xleftarrow{\$} \mathbb{Z}_q\}$, $\mathbf{e}_1 \leftarrow \{\text{Decomp}_{d_1}(\text{Comp}_{d_1}(x)) - x \mid x \xleftarrow{\$} \mathbb{Z}_q\}$, and $e_2 \leftarrow \{\text{Decomp}_{d_2}(\text{Comp}_{d_2}(x)) - x \mid x \xleftarrow{\$} \mathbb{Z}_q\}$. Using the central limit theorem, we further model all the coordinates in multiplication item $\mathbf{e}_t^T \cdot \mathbf{r} - \mathbf{e}_1^T \cdot \mathbf{s}$ as the Gaussian distribution $\mathcal{N}(0, n \cdot (\sigma_{\mathbf{e}_t}^2 \sigma_{\mathbf{r}}^2 + \sigma_{\mathbf{e}_1}^2 \sigma_{\mathbf{s}}^2))$. The distribution of e_2 is characterized by a discrete probability distribution table. We can first calculate the probability that the absolute value of a single coordinate of e_{total} exceeds $\frac{q-1}{4}$, and then derive the overall decryption failure rate via the union bound. The decryption failure rates of all parameter sets of COMPASS-PKE, namely COMPASS-KEM, is shown in Table 3.

Table 3: Decryption failure rates

Security Level	decryption failure rate
Level 1	$2^{-189.27}$
Level 2	$2^{-181.13}$
Level 3	$2^{-404.51}$
Level 4	$2^{-250.31}$

We note that the maximal number of decapsulation calls is bounded to 2^{80} in the proposal, so the decryption failure rates for all levels are far below the number of calls, which means that exploiting the decryption failure is nearly impossible.

3.4 Other Considerations

There are a few works [YXP20, XWG24] which claim that a kleptography attack may be applied to post-quantum KEM, especially in the key generation algorithm. For the [YXP20] attack, we include the master seed ι used in key generation into the secret key, so this attack can be detected by the secret key holder: first use ι to generate the public seed ρ , then check whether ρ has been modified in the public key.

In the post-quantum Kyber-KEM backdoor of Xia et al. [XWG24], the attacker embeds a McEliece ciphertext into the Least Significant Bits (LSBs) of the public-key component $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e}$, and later extracts these LSBs from the public key to recover the secret seed. This attack relies on two structural properties: (1) the LSBs of the public-key LWE sample must remain available, and (2) the key generation algorithm must contain an explicit error term whose parity can be controlled while preserving its prescribed distribution.

Our KEM design removes both attack surfaces. First, the public-key component is not published as the full value $\mathbf{t}'$. Instead, key generation outputs only $\mathbf{t} = \text{Comp}_d(\mathbf{t}')$,

so the d LSBs of each coefficient are discarded before serialization. The decompression procedure returns only a canonical rounded representative and does not reconstruct the original least significant bits. Therefore, the public key does not provide a reliable low-bit channel for embedding or recovering an auxiliary ciphertext. Second, the scheme follows the MLWR paradigm: the public-key noise is induced by deterministic rounding rather than by an independently sampled error vector. Hence, an adversarial key generator cannot encode a backdoor ciphertext by selecting the parity of an explicit error term while keeping the prescribed key distribution unchanged. Consequently, the LSB-based Kyber-KEM backdoor in [XWG24] is not applicable to our construction.

3.5 Dissimilarity Claim

Our scheme tightly follows the well-developed [LP11] paradigm with module lattices, which means that our scheme shares similar structures with other schemes under the same paradigm, typically ML-KEM and SABER. However, our scheme are significantly different from both of them.

Compared with ML-KEM, our scheme is based on different hard problems, namely MLWR and its variant, thus the error sampling in ML-KEM is replaced by a faster compression algorithm, that reduces the public key size as well as the total time cost of key generation and encapsulation algorithm, also avoids potential intellectual property issues in LWE-based schemes.

Compared with SABER, our scheme uses MLWR with prime moduli instead of power-of-2 moduli. MLWR with prime moduli can be reduced to MLWE, hence enjoys a worst-case to average-case reduction to standard lattice problems, while MLWR with power-of-2 moduli does not have a similar reduction. Moreover, using prime moduli allows us to perform NTT for polynomial multiplication, thus outperforms SABER in time complexity.

4. Security Claim and Analysis

4.1 Theoretical Security

In this part, we will show the IND-CPA security of our PKE scheme based on the decisional MLWR problem and biased MLWR problem by the following theorem:

4.1.1 IND-CPA Security

Theorem 2. *Let $p = (q - 1)/2^d$, $p' = (q - 1)/2^{d_1}$. For any adversary $\mathcal{A}$, there exist three adversaries $\mathcal{B}_0, \mathcal{B}_1$ and $\mathcal{B}_2$ with roughly the same running time as $\mathcal{A}$, such that*

$$Adv_{COMPASS.PKE}^{ind-cpa}(\mathcal{A}) \leq Adv_{\mathcal{H}()}^{prf}(\mathcal{B}_0) + Adv_{n,q,p,k,l}^{mlwr}(\mathcal{B}_1) + Adv_{n,q,p',d,k,l}^{biased-mlwr}(\mathcal{B}_2)$$

.

Proof. Since we set $d_2 \geq d_1$ in all parameters, we can assume that the adversary gets $\bar{c}_2 = \text{Comp}_{d_1}(c'_2)$ instead of c_2 , which only increases the advantage of an adversary.

The IND-CPA security of our PKE scheme can be proved by a sequence of security games that are indistinguishable by the security of PRF, the hardness of **MLWR** and the hardness of biased **MLWR**. The first game is the real IND-CPA game, and the final game satisfies that the public parameters are uniformly random and the ciphertext is generated

by applying a compression function to uniformly random elements, thus the advantage of the IND-CPA adversary is 0.

- **Game 0:** This is the real IND-CPA game that the public key and ciphertext are generated by the algorithms of MLWR-PKE.KeyGen and MLWR-PKE.Encrypt .
- **Game 1:** In this game, we change the way the public key is generated. Specifically, the public matrix $\hat{\mathbf{A}}$ is truly uniformly random, rather than being generated using a PRF.
- **Game 2:** In this game, we further change the way the public key is generated. Particularly, the public vector $\mathbf{t}'$ is selected uniformly at random, rather than being generated by algorithm MLWR-PKE.KeyGen .
- **Game 3** In this game, we change the way the ciphertext is generated, in which the elements $\mathbf{c}'_1$ and $\mathbf{c}'_2$ are selected uniformly at random.

Under the security of PRF, **Game 0** and **Game 1** are indistinguishable.

The **Game 1** and **Game 2** are indistinguishable by the hardness of **MLWR**.

For **Game 2** and **Game 3**, let $\mathbf{A}' = [\mathbf{A}|\mathbf{t}']$, then the concatenation of elements $\mathbf{c}_1$ and $\bar{\mathbf{c}}_2 = \text{Comp}_{d_1}(\mathbf{c}'_2)$ in **Game 2** is with the same distribution as $\mathbf{b} = \lfloor \mathbf{r}^T \mathbf{A}' + \mathbf{0}^T \rfloor \lfloor q/2 \rfloor \cdot m \rfloor_{p'}$, and from biased **MLWR**, it is independent with $\lfloor \mathbf{u} + \mathbf{0}^T \rfloor \lfloor q/2 \rfloor \cdot m \rfloor_{p'}$, $\mathbf{u}$ is uniformly random, which is equal to the distribution in **Game 3**. Then under biased **MLWR** assumption, **Game 2** and **Game 3** are indistinguishable. $\square$

This IND-CPA secure encryption scheme is the basis for the IND-CCA secure KEM $\text{.KEM} = (\text{Encaps}, \text{Decaps})$, which is obtained by using an appropriate transformation. Recently, several post-quantum versions [HHK17, JZC⁺18] of the Fujisaki-Okamoto transform with corresponding security reductions have been developed. At this point, the FO⁺ transformation in [HHK17] with post-quantum reduction from Jiang et al. [JZC⁺18] gives the tightest reduction for schemes with non-perfect correctness. However, other transformation could be used to turn .PKE into a CCA secure KEM.

4.1.2 IND-CCA Security in Classical Random Oracle

By modeling the hash functions $\mathcal{G}$ and $\mathcal{H}$ as random oracles, a lower bound on the CCA security can be proven. We use the security bound of Hofheinz et al. [HHK17], which considers a KEM variant of the Fujisaki-Okamoto transform that can also handle a small failure probability δ of the encryption scheme. This failure probability should be cryptographically negligibly small for the security to hold. From existing works such as [JZC⁺18], we get the following theorems for the security and correctness of our KEM in the random oracle model:

Theorem 3. *In the random oracle model, where $\mathcal{G}$ and $\mathcal{H}$ are assumed to be random oracles, for a IND-CCA adversary $\mathcal{B}$, making at most $q_{\mathcal{H}}$ and $q_{\mathcal{G}}$ queries to respectively the random oracle $\mathcal{G}$ and $\mathcal{H}$, and q_D queries to the decryption oracle, there exists an IND-CPA adversary $\mathcal{A}$ with approximately the same running time as adversary $\mathcal{B}$, such that:*

$$\text{Adv}_{\text{COMPASS.KEM}}^{\text{ind-cca}}(\mathcal{B}) \leq 3\text{Adv}_{\text{COMPASS.PKE}}^{\text{ind-cpa}}(\mathcal{A}) + q_{\mathcal{G}}\delta + \frac{2q_{\mathcal{G}} + q_{\mathcal{H}} + 1}{2^n}.$$

4.1.3 IND-CCA Security in Quantum Random Oracle

We can reduce the IND-CCA security of .KEM from the IND.CPA security of the underlying public key encryption.

Theorem 4. *In the quantum random oracle model, where $\mathcal{G}$ and $\mathcal{H}$ are assumed to be random oracles, for any IND-CCA quantum adversary $\mathcal{B}$, making at most $q_{\mathcal{H}}$ and $q_{\mathcal{G}}$ queries to respectively the random quantum oracle $\mathcal{G}$ and $\mathcal{H}$, and q_D many (classical) queries to the decryption oracle, there exists an adversary $\mathcal{A}$, with approximately the same running time as $\mathcal{B}$, such that:*

$$Adv_{COMPASS.KEM}^{ind-cca}(\mathcal{B}) \leq 2q_{\mathcal{H}} \frac{1}{\sqrt{2^n}} + 4q_{\mathcal{G}}\sqrt{\delta} + 2(q_{\mathcal{G}} + q_{\mathcal{H}})\sqrt{Adv_{COMPASS.PKE}^{ind-cpa}(\mathcal{A}) + 1/|M|}.$$

In all attack scenarios we assume that the depth of quantum computation is limited to 2^{64} quantum gates.

4.2 Concrete Security

Any MLWR instance can be converted into an LWR instance of size $n \cdot k \times n \cdot \ell$, i.e., with a secret vector of length $n_{LWR} = n \cdot \ell$ and $m_{LWR} = n \cdot k$ samples. In our discussion below in this section, we abuse the notations such that $(\mathbf{A}, \mathbf{t})$ stands for LWR instance instead of MLWR instance.

LWR can be considered as a variant of LWE. Specifically, we can write $\lfloor \frac{p}{q} \mathbf{A} \mathbf{s} \rfloor = \mathbf{t}$ in an equivalent form: $\mathbf{A} \mathbf{s} + \mathbf{e} = \bar{\mathbf{t}}$, where $\bar{\mathbf{t}} = \frac{q}{p} \mathbf{t} + \frac{q}{2p} \mathbf{1}$ (here $\frac{q}{2p} \mathbf{1}$, a vector whose all elements are $\frac{q}{2p}$, is used to recenter the mean value of $\mathbf{e}$ to zero, since we use $\lfloor \cdot \rfloor$ instead of $\lfloor \cdot \rceil$), and $\mathbf{e} = \bar{\mathbf{t}} - \mathbf{A} \mathbf{s}$ is called the rounding error, which can be viewed as the error vector in LWE. This is why that we can directly use the security estimation of LWE on LWR.

4.2.1 Primal Attack

Primal Attack [AGVW17] reduces the LWE problem to the unique Shortest Vector Problem (uSVP) using Kannan's embedding technique [Kan83], and then employs lattice reduction algorithms to recover the target vector. Albrecht et al. [AFG13] first proposed the primal attack against the LWE problem. This method can also be naturally extended to LWR by treating the rounding errors as the noise components to be recovered.

For the security against key recovery, the attack can be performed by finding the LWR secret given LWR sample $(\mathbf{A}, \mathbf{t})$, let $\tilde{\mathbf{t}} = \begin{pmatrix} \bar{\mathbf{t}} \\ \mathbf{0} \end{pmatrix}$, $\bar{\mathbf{t}}$ be defined as above. The basis of the embedded lattice is constructed as follows:

$$\mathbf{B} = \begin{pmatrix} \mathbf{A}' & \tilde{\mathbf{t}} \\ \mathbf{0}^T & 1 \end{pmatrix},$$

where $\mathbf{A}' = \begin{pmatrix} q\mathbf{I}_{m-n} & \mathbf{A} \\ \mathbf{0} & \mathbf{I}_n \end{pmatrix}$, In this embedded lattice $\mathbf{B}$, there exists a unique shortest lattice vector $\mathbf{v} = (\mathbf{e}, \mathbf{s}, 1)$, where $\mathbf{e}$ is the LWR rounding error. Its square norm is approximately $\|\mathbf{v}\|^2 \approx \sigma^2 \cdot m + \frac{q^2}{12p^2} \cdot n$, which is shorter than the second shortest lattice vector $\lambda_2(\mathcal{L})$. Therefore, the LWR problem is reduced to the uSVP problem on the embedded lattice. Security evaluation models for evaluating uSVP on the embedded lattice include: the Core-SVP model [ADPS16], the usvp model in the Lattice Estimator, and the two-step mode security evaluation model [XWW⁺24].

For the security against plaintext recovery, the (biased) LWR sample is given as: $(\mathbf{A} \parallel \mathbf{t}', \mathbf{c}_1 \parallel \mathbf{c}_2)$. The lattice basis can be constructed similarly, which can be considered as a LWR instance with more samples $(n \cdot (l + 1) > n \cdot k)$, since we set $k = l$ for all security levels). However, we claim that the optimal number of samples for solving LWR is below nk , thus adding more samples is not helpful for breaking the security of the scheme. So in our discussion below, we only consider the security level against key recovery.

The LWE security evaluation model in two-step mode [XWW⁺24] provides refined estimation for LWE hardness analysis, which can also be applied to LWR primal attack described above. Research indicates that compared to the standalone BKZ mode [SE94, CN11], the two-step mode can solve uSVP more efficiently, thereby optimizing the evaluation of LWR security. We present the concrete hardness level using the refined two-step estimator in the table below.

Table 4: Concrete Security Parameters from the Two-step Mode under Classical and Quantum Parameter Settings.[XWW⁺24]

Setting	Security Level	n_{LWR}	m_{LWR}	q	p	η_s	Memory($\log_2(\text{bit})$)	Time($\log_2(\text{gate})$)	d_{primal}	β_{BKZ}	β_{sieve}
Classical	128	512	512	3329	832	3	99.1	142.64	1025	398	459
Classical	256	1024	1024	3329	832	2	199.83	284.1	1912	902	969
Classical	384	1536	1536	7681	1920	3	288.77	409.94	2892	1348	1419
Classical	512	2048	2048	7681	1920	4	400.26	568.28	3849	1910	1982
Quantum	128	512	512	3329	832	3	99.82	123.62	1025	394	462
Quantum	256	1024	1024	3329	832	2	200.56	245.92	1912	897	972
Quantum	384	1536	1536	7681	1920	3	289.58	354.87	2892	1343	1423
Quantum	512	2048	2048	7681	1920	4	401.17	491.87	3849	1904	1986

β_{BKZ} : the largest blocksize β for BKZ preprocess.

Moreover, we compare the concrete security between our algorithm and ML-KEM. We show that our algorithm has higher time complexity than ML-KEM under similar security levels.

Table 5: Comparison of Concrete Security Parameters between COMPASS-KEM and ML-KEM under Primal Attack.

Security Level	Algorithm Name	Time($\log_2(\text{gate})$)	Memory($\log_2(\text{bit})$)
128-bit	COMPASS-KEM-128	142.6	99.1
128-bit	ML-KEM-512	141.4	98.1
256-bit	COMPASS-KEM-256	284.1	199.8
256-bit	ML-KEM-1024	276.9	194.6

To verify the security of the proposed scheme, we conducted concrete parameter evaluations for the LWR instances under traditional primal attack paths, such as uSVP and BDD. Experimental results demonstrate that, given the same security parameter settings, the computational overhead (**Time**) of traditional attack paths is higher than that of the **Two-step Mode** shown in Table 4. The evaluation framework based on the Two-step Mode provides a more accurate capture of security bottlenecks during lattice reduction, ensuring that the bit-security for each security level is robustly higher than the target thresholds.

4.2.2 Dual Attack

A recent work [CMHST25] has refined the MATZOV-style dual attack, and show that the hardness estimation of ML-KEM under their model is much lower than the primal attack. However, their estimation relies on many unstable heuristic assumptions, and is not known to be applicable to other schemes. For the sake of completeness, we also give the security estimation of our scheme under their model, and show that the estimation results exceed the required thresholds under all security levels.

Table 6: Concrete Security Parameters from the latest Dual Attack under Classical and Quantum Parameter Settings.

Setting	Security Level	n_{LWR}	m_{LWR}	q	p	η_s	Time($\log_2(\text{gate})$)	d_{dual}	n_{enum}	n_{fft}	k_{fft}	β_{BKZ}	β_{sieve}
Classical	128	512	512	3329	832	3	139.40	477	5	52	9	383	387
Classical	256	1024	1024	3329	832	2	259.44	830	10	137	19	810	791
Classical	384	1536	1536	7681	1920	3	383.06	1278	7	188	27	1249	1209
Classical	512	2048	2048	7681	1920	4	538.73	1889	6	249	39	1801	1735
Quantum	128	512	512	3329	832	3	123.58	482	1	45	8	387	397
Quantum	256	1024	1024	3329	832	2	228.36	853	4	125	17	822	809
Quantum	384	1536	1536	7681	1920	3	335.88	1294	10	165	23	1262	1228
Quantum	512	2048	2048	7681	1920	4	472.52	1886	5	224	34	1822	1762

According to the evaluation results presented in Table 6, the estimated cost of the coded-dual filtering stage is significantly lower than the corresponding primal-attack estimates. However, this comparison should be interpreted with caution. Specifically, the result shows that our parameter choices for 384-bit security level has only 383.06-bit security, that falls below the requirement threshold. However, we claim that the [CMHST25] estimation allows a success rate of only $\approx 31\%$, which means that the real security bit should be at least $\log_2(\frac{1}{0.31}) \approx 1.69$ bit higher than the estimation result, so our parameter for 384-bit security can still meet the requirement.

Additionally, the coded-dual cost model assumes that the final sub-LWE solving step is negligible, whereas in an actual key-recovery setting this step may require solving a residual LWE instance of dimension $n - n_{\text{enu}}$. If the residual instance is solved by a primal attack, its cost should not be ignored, especially when n_{enu} is small. Consequently, although the reported dual-attack estimate is substantially lower, the model may underestimate the full cost of complete secret recovery.

5. Performances

In this section, we provide a comprehensive evaluation of the performance characteristics of the proposed COMPASS-KEM. The assessment is structured into two primary parts: first, a theoretical performance analysis is presented in Section 5.1, where we discuss the implementational tradeoffs and the efficiency gains derived from the MLWR mechanism—most notably the elimination of explicit noise sampling. Second, in Section 5.2, we report the quantitative benchmarking results obtained from our ANSI C reference implementation. By evaluating the cycle counts across all designated security levels, we aim to demonstrate that COMPASS-KEM achieves a superior balance between computational efficiency and cryptographic robustness, making it suitable for a wide range of practical applications.

5.1 Performance Analysis

In this section, we consider the implementational aspects of COMPASS-KEM and discuss various design tradeoffs. It is important to note that the current performance evaluation is based entirely on our ANSI C reference implementation. While platform-specific optimizations, such as SIMD vectorization or hardware acceleration, are left for future work, the fundamental design of COMPASS-KEM ensures high efficiency even in a strictly baseline software environment.

5.1.1 Implementation Considerations and Tradeoffs

Implementing the NTT. Many different tradeoffs are possible when implementing the number-theoretic transform, primarily balancing code size and execution speed. Our C reference implementation of COMPASS-KEM utilizes dedicated forward NTT and inverse INTT algorithms, leveraging precomputed tables of powers of ζ to avoid repetitive on-the-fly calculations. A key advantage of using the NTT on memory-constrained platforms is that the multiplication of two elements in R_q can be computed in-place, without requiring any additional temporary storage. Although our current implementation relies on standard sequential C code, the NTT structure is highly amenable to future vectorization.

Symmetric Primitives. The most compute-intensive components inside COMPASS-KEM are the symmetric primitives used for hashing and pseudorandom number generation. In a pure C implementation without parallel vector instructions, sequential hashing can be a significant performance bottleneck. However, COMPASS-KEM gains a crucial architectural advantage over MLWE-based schemes in this exact scenario: by relying on the deterministic rounding mechanism, we completely eliminate the need to sample explicit noise vectors (such as CBD sampling for error terms). This reduction in required PRF invocations noticeably lowers the symmetric cryptography overhead, allowing our unoptimized C implementation to remain highly competitive.

Tradeoffs between secret-key size and speed. Implementers can choose different tradeoffs between the size of the secret key and the computational latency of decapsulation. If minimizing the secret-key size is paramount, it is possible to omit storing the public key pk and its hash $H(pk)$ within the secret key, and instead recompute them during decapsulation. Furthermore, since all randomness during key generation is derived from initial seeds, one could theoretically store only the 32-byte core seed and re-run the entire key generation process during decapsulation, although this significantly increases computational time.

Conversely, if secret-key size is not a strict constraint and decapsulation speed is the priority, one might decide to store the fully expanded matrix $\hat{\mathbf{A}}$ as part of the secret key. This avoids the expensive recomputation of $\hat{\mathbf{A}}$ from the seed ρ during the re-encapsulation phase of the Fujisaki-Okamoto (FO) transform. Our reference implementation adopts a balanced approach: we store the polynomials in the NTT domain along with the public key and $H(pk)$, but we do not cache the expanded matrix $\hat{\mathbf{A}}$ to maintain a reasonable key size.

Local storage format of static public keys. For applications where a user frequently encapsulates messages to the same static public key, the encapsulation process can be further accelerated by locally caching the expanded public key, which includes the matrix $\hat{\mathbf{A}}$ and $H(pk)$. This saves the substantial cost of expanding the matrix $\hat{\mathbf{A}}$ from the seed ρ and the cost of hashing pk during every single encapsulation operation.

5.1.2 Performance Scaling Across Security Levels

The computational overhead of COMPASS-KEM exhibits a deterministic scaling pattern across the four designated security levels (from 128-bit to 512-bit security). The cycle counts for Key Generation, Encapsulation, and Decapsulation naturally increase alongside the security parameters, driven primarily by the expansion of the matrix dimensions ($k \times l$) and the polynomial degree (n).

Impact of Matrix Dimensions ($k \times l$). The most compute-intensive operation in both KeyGen and Encaps algorithms is the generation of the pseudorandom matrix $\hat{\mathbf{A}} \in R_q^{k \times l}$ using an Extendable-Output Function (XOF) based on the hash. Because the number of polynomial entries in $\hat{\mathbf{A}}$ grows proportionally to $k \times l$, the XOF squeezing cycles scale quadratically with the dimension parameter. For instance, transitioning from Level 1 ($k = l = 2$) to Level 2 ($k = l = 4$) while keeping the degree $n = 256$ constant results in approximately a fourfold increase in the matrix expansion overhead.

Impact of Polynomial Degree (n). For Level 3 and Level 4, the polynomial degree doubles from $n = 256$ to $n = 512$, and the modulus q expands from 3329 to 7681. This directly impacts the Number-Theoretic Transform (NTT) and inverse INTT operations. Since the asymptotic complexity of the NTT is $O(n \log n)$, doubling the degree results in slightly more than double the polynomial arithmetic overhead per vector entry. Furthermore, the larger modulus and extended polynomial length double the data memory footprint. This exerts higher pressure on the CPU’s cache subsystem (L1/L2 bandwidth), causing the performance scaling curve to become slightly non-linear due to increased memory access latency during large-scale matrix-vector multiplications.

The MLWR Scaling Advantage. A critical observation from the scaling trend is the mitigation of symmetric cryptography bottlenecks at higher security levels. In traditional MLWE-based KEMs, scaling to higher security levels necessitates sampling correspondingly larger error vectors ($\mathbf{e}_1, \mathbf{e}_2$) via a PRF, which incurs a substantial and growing hash overhead. Because our COMPASS-KEM design inherently eliminates explicit noise sampling through deterministic rounding, the generation of the error terms requires zero PRF invocations. Therefore, even as we scale to the most demanding Level 4 (512-bit quantum security, $k = 4, n = 512$), the symmetric performance penalty remains strictly confined to the expansion of $\hat{\mathbf{A}}$ and transcript hashing. This keeps the overall cycle count significantly lower and the scaling curve flatter compared to analogous MLWE schemes.

5.2 Reference and Optimized Implementations

In accordance with cryptographic algorithm submission guidelines, we provide both a highly portable Reference Implementation and a Platform-Optimized Implementation.

5.2.1 Portable ANSI C Baseline

To evaluate the baseline efficiency of COMPASS-KEM, we first benchmarked our pure C reference implementation. The performance profiling was conducted on a workstation equipped with an **AMD Ryzen 7 8745HS** processor (4 cores) running **Ubuntu 22.04**. The source code was compiled using **GCC 11.4.0** with the **-O3** optimization flag. To ensure the reliability of the cycle count measurements, CPU dynamic frequency scaling was disabled, the process was pinned to a single core, and the reported clock cycles were averaged to mitigate OS-level noise.

5.2.2 AVX2 Optimized Implementation

To complement our portable baseline, we developed a highly optimized implementation targeting modern x86 processors equipped with the Advanced Vector Extensions 2 (AVX2) instruction set. The comparative performance results are detailed in Table 7. Instead of relying on generic optimization frameworks, we engineered a bespoke AVX2 architecture tailored specifically to the unique algebraic structures of our scheme. This approach fully exploits data-level parallelism to accelerate both the symmetric cryptographic primitives and the core polynomial arithmetic autonomously.

Table 7: Performance Comparison of KEM Algorithms (Clock Cycles on x86 CPU)

Algorithm	Security (bits)	KeyGen	Encaps	Decaps
ML-KEM-512	128	134,556	159,329	206,350
COMPASS-KEM-128 (C Ref)	128	121,634	161,229	216,570
COMPASS-KEM-128 (AVX2)	128	32,887	28,080	36,118
ML-KEM-1024	256	329,278	369,079	457,594
COMPASS-KEM-256 (C Ref)	256	317,468	387,720	446,243
COMPASS-KEM-256 (AVX2)	256	53,081	55,933	65,690
COMPASS-KEM-384 (C Ref)	384	572,264	826,889	1,093,750
COMPASS-KEM-384 (AVX2)	384	141,590	137,521	166,334
COMPASS-KEM-512 (C Ref)	512	871,519	1,178,215	1,412,591
COMPASS-KEM-512 (AVX2)	512	198,000	183,184	234,086

Vectorized Symmetric Primitives A significant portion of the computational latency during the **KeyGen** and **Encaps** procedures is consumed by the extendable-output function (XOF) used for the expansion of the public matrix $\hat{\mathbf{A}}$. To mitigate this bottleneck regardless of the underlying cryptographic standard, our AVX2 implementation employs a generalized parallel hashing methodology. By leveraging the 256-bit `_m256i` registers, we evaluate multiple independent hash states concurrently. The specific multi-way parallelization factor is dynamically aligned with the internal word size of the deployed symmetric standard. This interleaved execution architecture minimizes instruction overhead and dramatically reduces the latency of matrix generation by fully saturating the wide execution pipelines of modern x86 processors.

Dual-Parameter Polynomial Arithmetic and NTT A distinctive engineering highlight of our AVX2 implementation is its native support for our dual-modulus architecture. To cover all four security levels, COMPASS-KEM operates on two distinct parameter sets: $q = 3329$ with $n = 256$ for Levels 1 and 2, and $q = 7681$ with $n = 512$ for Levels 3 and 4.

Since both moduli safely fit within 13 bits of precision, the polynomial coefficients are compactly represented as 16-bit signed integers (`int16_t`). Consequently, a single 256-bit AVX2 register packs exactly 16 coefficients, achieving a highly efficient $16\times$ parallelization factor. To maximize instruction throughput without introducing branch penalties, we independently designed two separate, highly unrolled sets of AVX2 intrinsic codes tailored to each modulus:

- **For $q = 3329, n = 256$:** We constructed a dedicated set of vectorized routines for the 8-layer Number-Theoretic Transform (NTT). The Cooley-Tukey butterfly operations leverage the `_mm256_mullo_epi16` and `_mm256_mulhi_epi16` instructions to execute 16-bit modular arithmetic natively within the vector registers.
- **For $q = 7681, n = 512$:** For this high-security parameter set, we engineered advanced 16-bit SIMD memory routing and shuffle instructions (e.g., `_mm256_shuffle_epi32` and `_mm256_blend_epi32`) to seamlessly handle the larger polynomial arrays across a 9-layer NTT, effectively mitigating L1 cache-miss penalties.

Lazy Reduction and Layer Merging. A major algorithmic optimization in our AVX2 NTT/INTT is the application of *lazy reduction*. Instead of performing modular reduction after every single butterfly layer, we exploit the 16-bit signed integer boundary (32,767). By meticulously tracking the coefficient growth, we safely accumulate values up to $4q$ across up to three fused butterfly layers within the AVX2 registers. A strict vectorized Barrett reduction is only enforced immediately prior to memory write-back. This strategy drastically reduces the total instruction count and pipeline latency. Extensive randomized testing confirms that this optimized SIMD arithmetic yields bit-for-bit identical results to our portable C reference implementation.

Public Key Compression and Data Packing Unlike Fiat-Shamir signature schemes that rely heavily on rejection sampling, the efficiency of our COMPASS-KEM is intrinsically tied to the performance of its deterministic rounding mechanisms (the `Compd` and `Decompd` functions). The AVX2 instruction set is critically instrumental in accelerating these operations. We employ 16-bit vectorized addition, subtraction, and arithmetic shift instructions (e.g., `_mm256_srai_epi16`) to execute the rounding process in a strictly branchless manner. This autonomous mechanism allows our algorithm to compress the ciphertext and public key coefficients instantly without pipeline stalls. Furthermore, the bit-packing and unpacking routines required for serialization are fully vectorized from the ground up, replacing sequential bitwise loops with high-speed SIMD shifts, blends, and masks.

Future Hardware Acceleration Prospects While the current AVX2 implementation demonstrates exceptional software efficiency, the structured parallelism of COMPASS-KEM makes it inherently adaptable to heterogeneous architectures. Future work will explore hardware-software co-design methodologies—such as offloading the symmetric primitives and NTT operations to Field Programmable Gate Arrays (FPGAs) or leveraging emerging RISC-V Vector (RVV) extensions. This will further maximize throughput and minimize energy consumption for high-performance computing constraints.

6. Feature Statement

In this section, we make a statement on the features of our algorithm.

- *Novelty.* Our algorithmic proposal is based on the MLWR and a variant of the MLWR assumption. Regarding the MLWR assumption, unlike some proposals in the existing NIST standardization process, such as Saber and Round 5, the parameters we choose satisfy the reduction requirements in [LW20], i.e., there exists a theoretical hardness lower bound. By proposing a newly proposed variant of MLWR, namely

the biased MLWR assumption, we fix the flawed security proof in the documentation of SABER [DKRV18]. For the new problem, we provide a rigorous definition and, in particular, establish for the first time a reduction from the standard MLWE problem to this problem (although the reduction employs known techniques). In this respect, we have achieved a notable innovation in the hardness foundation of our algorithm.

- *Simplicity.* Our scheme satisfies simplicity from design and implementation. Our scheme follows the standard LP paradigm for lattice-based encryption and QFO transformation for post-quantum KEM, both are well studied. We only use hard problems on module lattices with prime modulus, which can be reduced to standard MLWE that enjoys worst-case security, and the concrete security of lattice-based hard problems is also well-studied. For implementation, we use only basis operations including NTT, mod-mult and hash functions.
- *Flexibility.* Expect for the underlying cryptographic hash functions, the implementation of our scheme relies only on mod-integer operations, which can be efficiently implemented on both software and hardware platforms, and support a high level of parallelism, that allows a flexible balance between efficiency and resources.
- *Compatibility.* We also present a digital signature scheme inside the COMPASS suite, both schemes are based on module-lattices with same polynomial degrees (although the moduli are different). Combining COMPASS-KEM and COMPASS-SIG allows us to achieve other functionalities, including authenticated key exchange. Moreover, our scheme has shorter public key and ciphertext sizes than ML-KEM, so fitting our scheme into existing protocols could be easier than ML-KEM.
- *Extensibility.* Lattice-based schemes are known to be highly functional, including identity-based encryption, attribute-based encryption, inner-product encryption and fully homomorphic encryption. Our scheme can serve as a construction basis for those more advanced schemes.
- *Advantages of Implementation Performance across Platforms.* The basic operation unit of our scheme is polynomial in $R_q/(X^n + 1)$ for $n = 256$ or $n = 512$, which is relatively small compared with schemes based on RLWE or NTRU. While both NTT and mod-mult operations can be easily implemented in parallel, NTT operations on 256 or 512 dimensional vectors are also acceptable for the resource-restricted cases.

References

- [ADPS16] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key {Exchange—A} new hope. In *25th USENIX security symposium (USENIX Security 16)*, pages 327–343, 2016.
- [AFG13] Martin R. Albrecht, Robert Fitzpatrick, and Florian Goepfert. On the efficacy of solving LWE by reduction to unique-SVP. Cryptology ePrint Archive, Paper 2013/602, 2013.
- [AGVW17] Martin R. Albrecht, Florian Göpfert, Fernando Virdia, and Thomas Wunderer. Revisiting the expected cost of solving usvp and applications to lwe. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology*

- *ASIACRYPT 2017*, pages 297–322, Cham, 2017. Springer International Publishing.
- [CMHST25] Kevin Carrier, Charles Meyer-Hilfiger, Yixin Shen, and Jean-Pierre Tillich. Assessing the impact of a variant of matzov’s dual attack on kyber. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, pages 444–476, Cham, 2025. Springer Nature Switzerland.
- [CN11] Yuanmi Chen and Phong Q Nguyen. Bkz 2.0: Better lattice security estimates. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 1–20. Springer, 2011.
- [DKRV18] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-lwr based key exchange, cpa-secure encryption and cca-secure KEM. In Antoine Joux, Abderrahmane Nitaj, and Tajjeeddine Rachidi, editors, *Progress in Cryptology - AFRICACRYPT 2018 - 10th International Conference on Cryptology in Africa, Marrakesh, Morocco, May 7-9, 2018, Proceedings*, Lecture Notes in Computer Science, pages 282–305. Springer, 2018.
- [HHK17] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the fujisaki-okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography - 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12-15, 2017, Proceedings, Part I*, Lecture Notes in Computer Science, pages 341–371. Springer, 2017.
- [JZC⁺18] Haodong Jiang, Zhenfeng Zhang, Long Chen, Hong Wang, and Zhi Ma. Ind-cca-secure key encapsulation mechanism in the quantum random oracle model, revisited. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part III*, Lecture Notes in Computer Science, pages 96–125. Springer, 2018.
- [Kan83] Ravi Kannan. Improved algorithms for integer programming and related lattice problems. In *Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing*, STOC ’83, page 193–206, New York, NY, USA, 1983. Association for Computing Machinery.
- [LP11] Richard Lindner and Chris Peikert. Better key sizes (and attacks) for lwe-based encryption. In Aggelos Kiayias, editor, *Topics in Cryptology - CT-RSA 2011 - The Cryptographers’ Track at the RSA Conference 2011, San Francisco, CA, USA, February 14-18, 2011. Proceedings*, volume 6558 of *Lecture Notes in Computer Science*, pages 319–339. Springer, 2011.
- [LW20] Feng-Hao Liu and Zhedong Wang. Rounding in the rings. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part II*, Lecture Notes in Computer Science, pages 296–326. Springer, 2020.

- [SE94] Claus-Peter Schnorr and Martin Euchner. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical programming*, 66(1):181–199, 1994.
- [XWG24] Wenwen Xia, Geng Wang, and Dawu Gu. Post-quantum backdoor for kyberkem. In Maria Eichlseder and Sébastien Gambs, editors, *Selected Areas in Cryptography - SAC 2024 - 31st International Conference, Montreal, QC, Canada, August 28-30, 2024, Revised Selected Papers, Part I*, Lecture Notes in Computer Science, pages 237–255. Springer, 2024.
- [XWW⁺24] Wenwen Xia, Leizhang Wang, Geng Wang, Dawu Gu, and Baocang Wang. A refined hardness estimation of lwe in two-step mode. In *IACR International Conference on Public-Key Cryptography*, pages 3–35. Springer, 2024.
- [YXP20] Zhaomin Yang, Tianyuan Xie, and Yanbin Pan. Lattice klepto revisited. In Hung-Min Sun, Shiuh-Pyng Shieh, Guofei Gu, and Giuseppe Ateniese, editors, *ASIA CCS '20: The 15th ACM Asia Conference on Computer and Communications Security, Taipei, Taiwan, October 5-9, 2020*, pages 867–873. ACM, 2020.